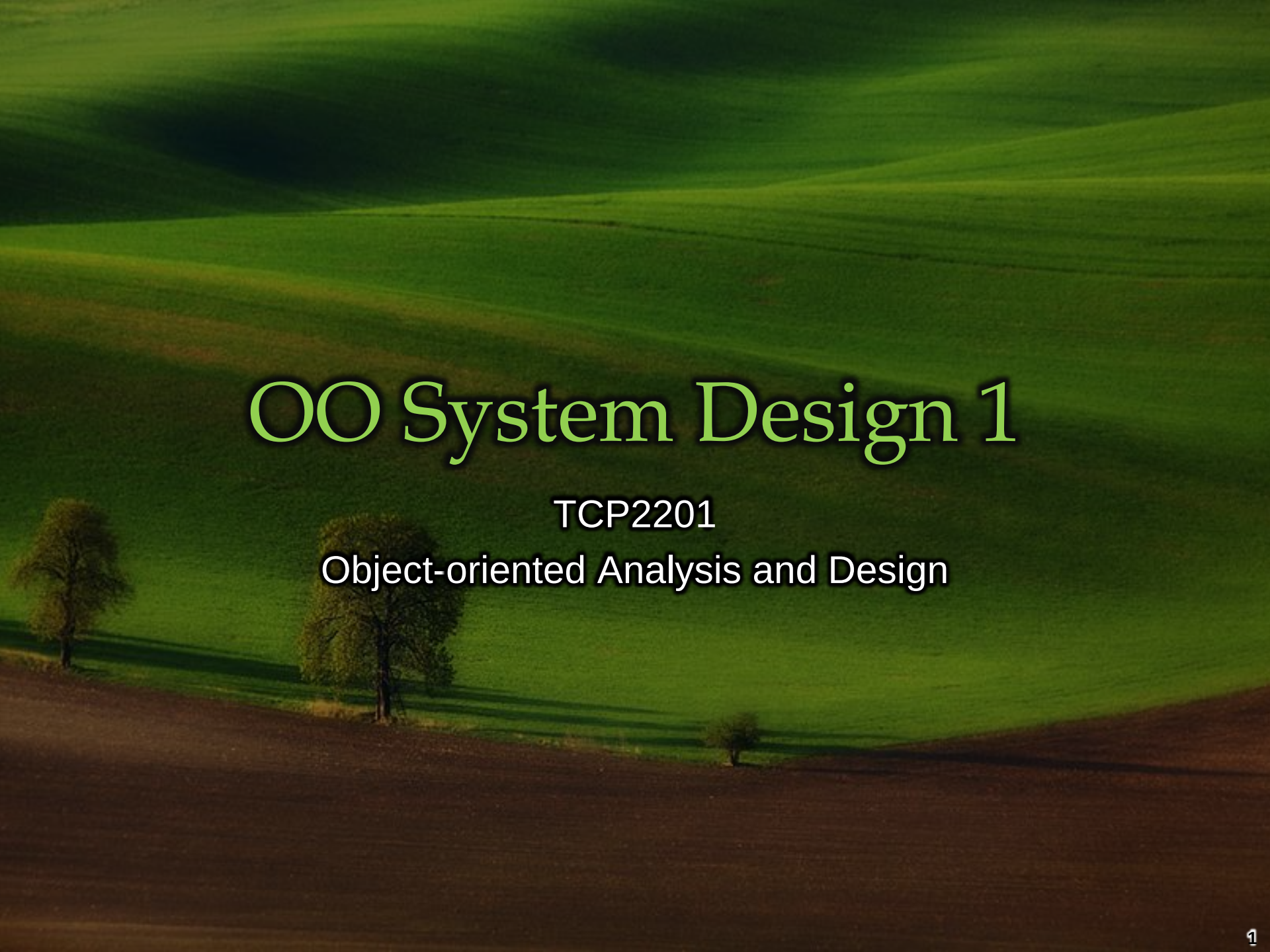




OO System Design 1

TCP2201

Object-oriented Analysis and Design

Lecture outcomes

- At the end of this lecture you should
 - understand how the design process takes over from the analysis process to create proper software designs
 - know the functions of a design document and how it helps to maintain control and structure in any software project.
 - learn about the outcomes from bad software design

The solution design process

- Recall that software engineering is

The process of solving customers' problems by the systematic development and evolution of large, high-quality software systems within cost, time and other constraints

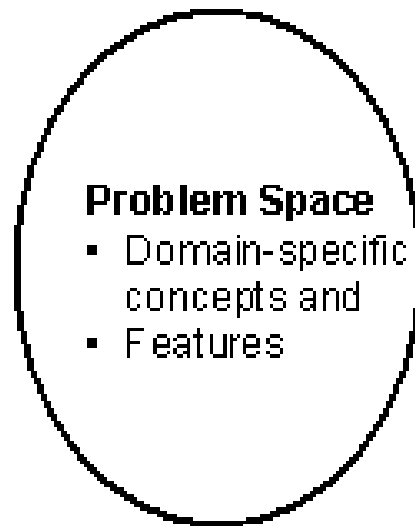
- To do this, software design is a problem-solving process whose objective is to find and describe a way:
 - To implement a system's functional requirements
 - While respecting the constraints imposed by the quality, platform and process requirements (including the budget)
 - And while adhering to general principles of good quality
- The design process takes the requirements obtained from domain analysis and formulates a solution that can be implemented (for our case, in software since this is a software course)
- We will focus on implementing OOP paradigms in software design

Design decisions

- A designer is faced with a series of *design issues*
 - These are sub-problems of the overall design problem.
 - Each issue normally has several alternative solutions:
 - these are the design *options* and it may not be possible to know which is the final one to be implemented
 - The designer makes a *design decision* to resolve each issue.
 - This process involves choosing the best option from among the alternatives.
- To make each design decision, the software engineer uses knowledge of
 - the requirements
 - the design as created so far
 - the technology available
 - software design principles and 'best practices'
 - what has worked well in the past ← this needs experience!

Design spaces

- Software design focuses on two distinct (but not independent) spaces – problem and solution
- The problem space was covered as the OO analysis last week



Domain specific
languages (DSLs)

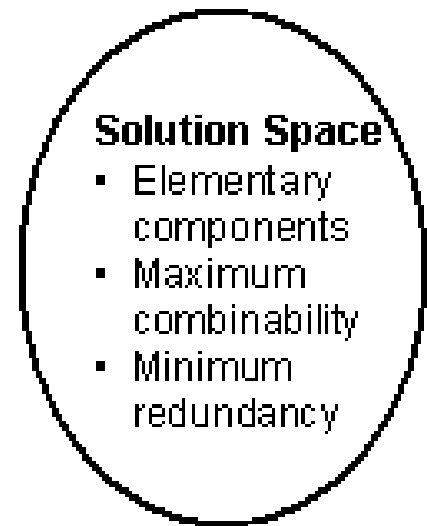
A large arrow pointing from the Problem Space to the Solution Space, containing the text 'Configuration Knowledge' and a bulleted list: 'Illegal feature combinations', 'Default settings', 'Default dependencies', 'Construction rules', and 'Optimizations'.

Configuration Knowledge

- Illegal feature combinations
- Default settings
- Default dependencies
- Construction rules
- Optimizations

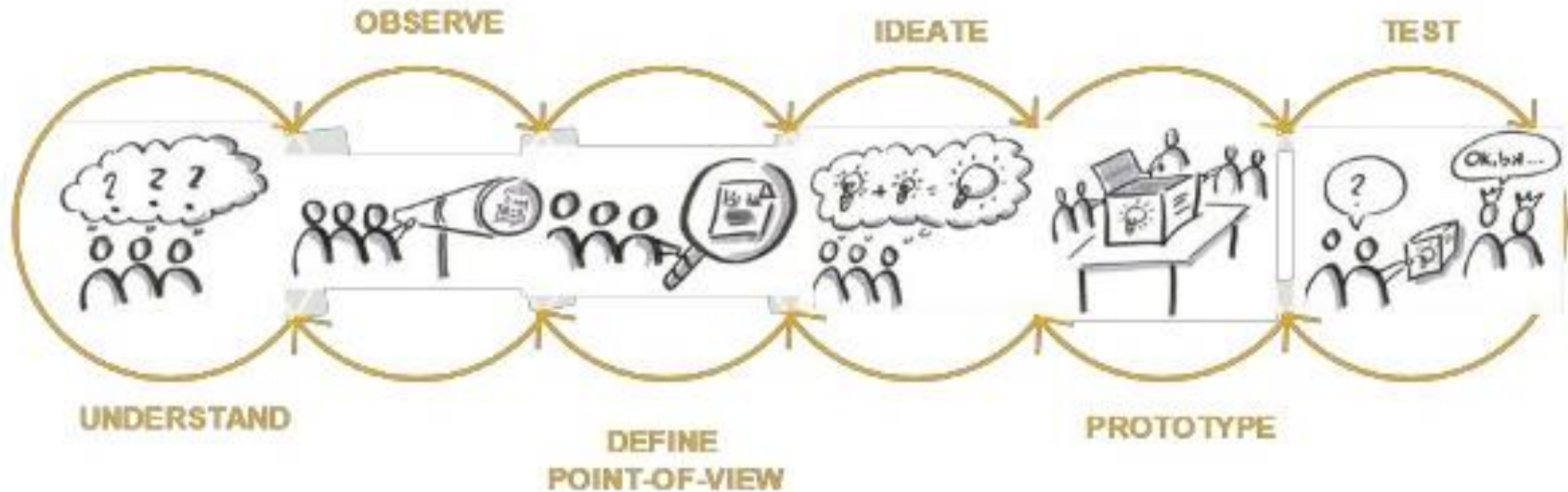
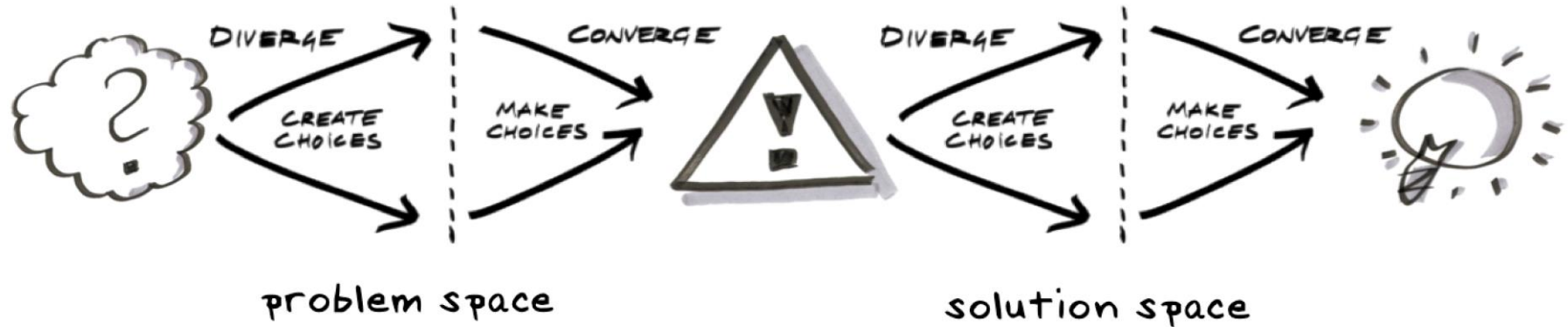
- Illegal feature combinations
- Default settings
- Default dependencies
- Construction rules
- Optimizations

Generators,
interpreters,
reflection, ...



Components +
system family
architecture

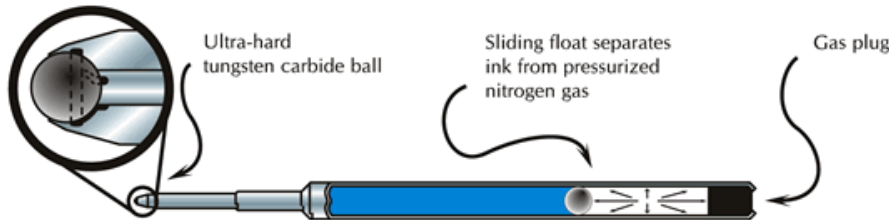
Mapping problem to solution spaces



Famous mapping problems

- The best problem analysis may not map to the best solution design

The Fisher Space Pen Sealed Pressurized Ink Cartridge



Stainless steel, precision-machined socket prevents leaks and oozing, yet delivers instant uniform ink flow

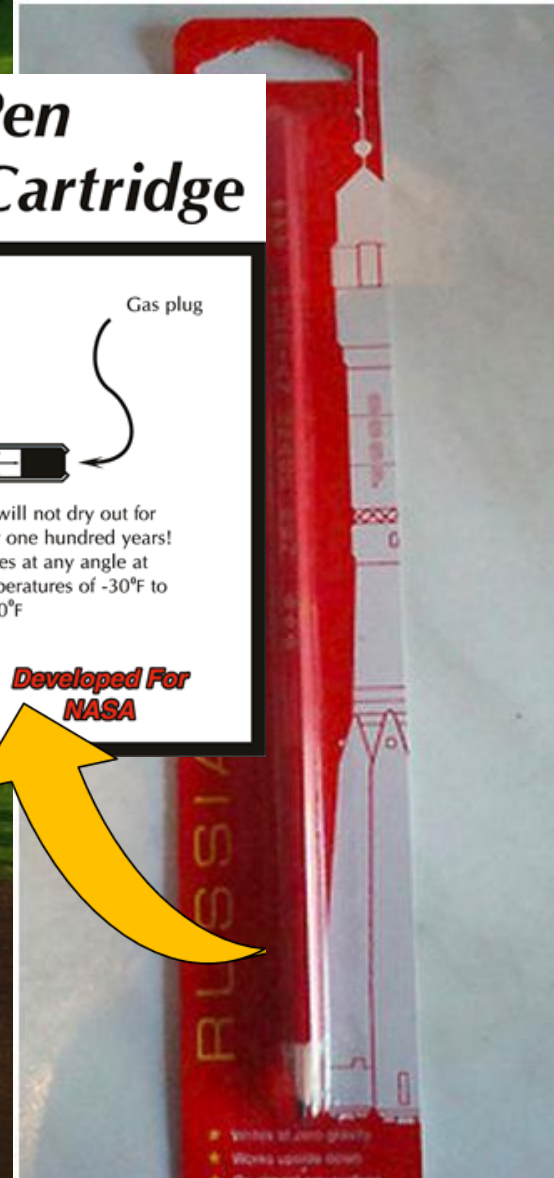
Thixotropic ink in a hermetically sealed and pressurized reservoir writes three times longer

Ink will not dry out for over one hundred years! Writes at any angle at temperatures of -30°F to +250°F

**Developed For
NASA**



MYTHBUSTING! NASA also used pencils during early space flights but due to tender supply deals, they were charged over US\$120 per pencil. To find a better option, NASA opted to find a non-flammable writing device. Fisher pens invested their own money to develop the FSP which was then offered to NASA at US\$2.39 per pen



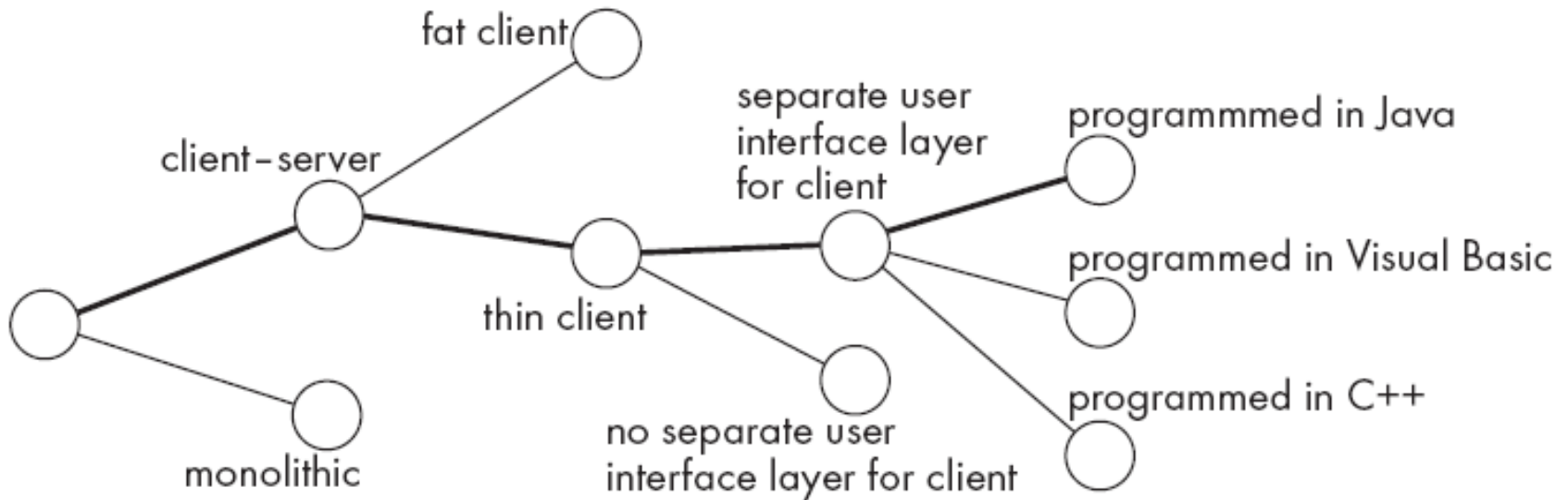
When NASA first started sending astronauts into space, they realized that the ball-point pen would not work at zero gravity.

A million dollar investment and two years of tests resulted in a pen that could write in space, upside down, on almost any surface and at temperatures ranging from below freezing to over 300°C.

When confronted with the same problem, the Russians used a pencil.

The solution design space

- The space of possible designs that could be achieved by choosing different sets of alternatives is often called the design space
- The outcome is the choice for deployment/implementation as the delivered system



OO Systems

- An OO system is a logical entity with a set of definable responsibilities or objectives consisting of hardware, software or both.
 - A system can have a specification which is then implemented by a collection of components.
 - A system continues to exist, even if its components are changed or replaced.
 - The goal of requirements analysis is to determine the responsibilities of a system.

Subsystem:

- A system that is part of a larger system, and which has a definite interface

Components and modules

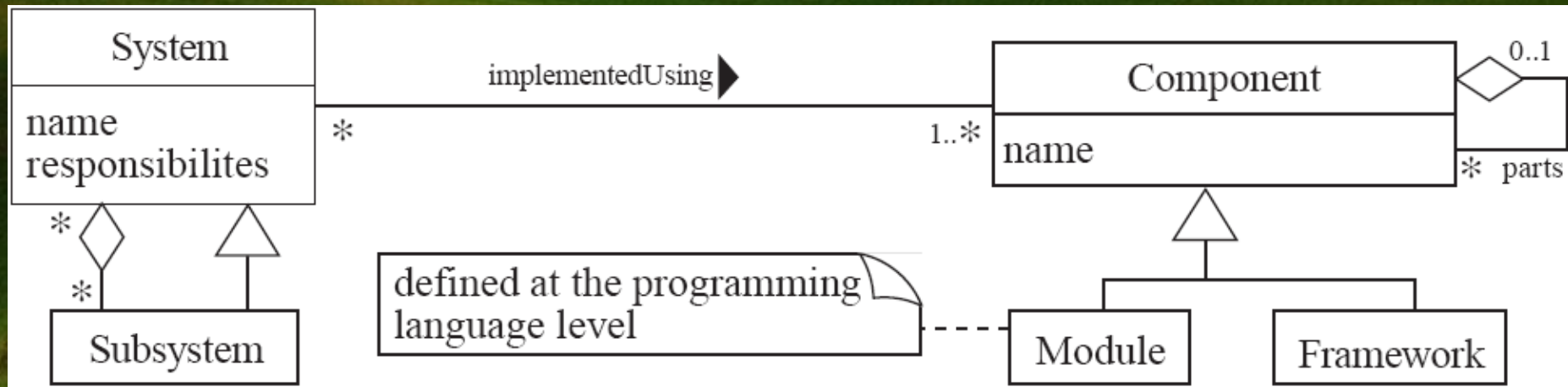
Component

- Any piece of software or hardware that has a clear role.
 - A component can be isolated, allowing you to replace it with a different component that has equivalent functionality.
 - Many components are designed to be reusable.
 - Conversely, others perform special-purpose functions.

Module

- A component that is defined at the programming language level
 - For example, methods, classes and packages are modules in Java.

UML diagram of system parts



Top-down and bottom-up design

Top-down design

- First design the high level structure of the system.
- Then gradually work down to detailed decisions about low-level constructs
- Finally arrive at detailed decisions e.g. format of data items, individual algorithms that will be developed



Bottom-up design

- Make decisions about reusable low-level utilities.
- Then decide how these will be put together to create high-level constructs to solve the problem space

A mix of top-down and bottom-up approaches are normally used:

- Top-down design is almost always needed to give the system a good structure.
- Bottom-up design is normally useful so that reusable components can be created.

Different aspects of design

- Architecture design:
 - The division into subsystems and components,
 - How these will be connected.
 - How they will interact.
 - Their interfaces.
- Class design:
 - The various features of classes.
- User interface design
- Algorithm design:
 - The design of computational mechanisms.
- Protocol design:
 - The design of communications protocol.

Why do you need good design?

- Overall goals of good design:
 - Increasing profit by reducing cost and increasing revenue
 - Ensuring that we actually conform with the requirements
 - Accelerating development
 - Increasing qualities such as
 - Usability
 - Efficiency
 - Reliability
 - Maintainability
 - Reusability

Design by contract

- A technique that allows you to design defensively in an efficient and systematic way
 - Key idea
 - each method has an explicit *contract* with its callers
 - The contract has a set of assertions that state:
 - What *preconditions* the called method requires to be true when it starts executing
 - What *postconditions* the called method agrees to ensure are true when it finishes executing
 - What *invariants* the called method agrees will not change as it executes

Making good design decisions

- Using priorities and objectives to decide among alternatives
 - Step 1: List and describe the alternatives for the design decision.
 - Step 2: List the advantages and disadvantages of each alternative with respect to your objectives and priorities.
 - Step 3: Determine whether any of the alternatives prevents you from meeting one or more of the objectives.
 - Step 4: Choose the alternative that helps you to best meet your objectives.
 - Step 5: Adjust priorities for subsequent decision making

Example

- Imagine a system has the following objectives, starting with top priority:
 - Security: Encryption must not be breakable within 100 hours of computing time on a 400Mhz Intel processor, using known cryptanalysis techniques.
 - Maintainability. No specific objective.
 - CPU efficiency. Must respond to the user within one second when running on a 400MHz Intel processor.
 - Network bandwidth efficiency: Must not require transmission of more than 8KB of data per transaction.
 - Memory efficiency. Must not consume over 20MB of RAM.
 - Portability. Must be able to run on Windows 98, NT 4 and ME as well as Linux

Evaluation of alternatives

	<i>Security</i>	<i>Maintainability</i>	<i>Memory efficiency</i>	<i>CPU efficiency</i>	<i>Bandwidth efficiency</i>	<i>Portability</i>
Algorithm A	High	Medium	High	Medium; DNMO	Low	Low
Algorithm B	High	High	Low	Medium; DNMO	Medium	Low
Algorithm C	High	High	High	Low; DNMO	High	Low
Algorithm D	—	—	—	Medium; DNMO	DNMO	—
Algorithm E	DNMO	—	—	Low; DNMO	—	—

Alternatives

- If all methods fail to find best design, use cost-benefit analysis to choose among alternatives instead
 - To estimate the costs, add up:
 - The incremental cost of doing the *software engineering* work, including ongoing maintenance
 - The incremental costs of any *development technology* required
 - The incremental costs that *end-users and product support personnel* will experience
 - To estimate the *benefits*, add up:
 - The incremental software engineering time saved
 - The incremental benefits measured in terms of either increased sales or else financial benefit to users

Design documentation

- Design documents as an aid to making better designs
- They force you to be explicit and consider the important issues before starting implementation.
- They allow a group of people to review the design and therefore to improve it.
- Design documents as a means of communication.
 - To those who will be implementing the design.
 - To those who will need, in the future, to modify the design.
 - To those who need to create systems or subsystems that interface with the system being designed.

Structure of a design document

- Purpose:
 - What system or part of the system this design document describes.
 - Make reference to the requirements that are being implemented by this design (traceability).
- General priorities:
 - Describe the priorities used to guide the design process.
- Outline of the design:
 - Give a high-level description of the design that allows the reader to quickly get a general feeling for it.
- Major design issues:
 - Discuss the important issues that had to be resolved.
 - Give the possible alternatives that were considered, the final decision and the rationale for the decision.
- Other details of the design:
 - Give any other details the reader may want to know that have not yet been mentioned.

- Avoid documenting information that would be readily obvious to a skilled programmer or designer → write the document to the level of the intended reader.
- Avoid writing details in a design document that would be better placed as comments in the code.
- Avoid writing details that can be extracted automatically from the code, such as the list of public methods.

Example

- Examples of design documents

Design Document

Template

[Company Name]
[Project Name]

2 System Overview

Provide a brief overview of the system. Ensure that this includes:

NOTE: Highlight more in the High-Level Design doc.

Label each component so that they may reference code and spreadsheets when referencing subsystems and components.

2.1 System Information

Specify the Database Management System, configuration utilities, and any support software used.

2.1.1 Database Management System

Identify the vendor, version and target hardware for restrictions on the initialization and use of the DBMS.

Vendor	Hardware

2.1.2 Database Software Utilities

Identify any utility software that will be used to support

Vendor	Product

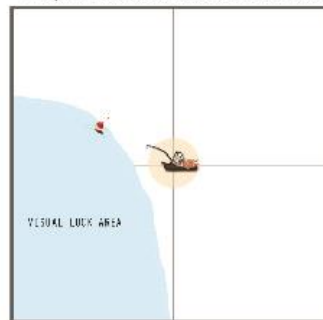
Game Design Document

Abstract

This is a rehabilitation game for the patients who have visual field defect after brain injury. The concept is that the patients will get mysterious and miraculous fish and correct the fishes to their own aquarium by the rehabilitation game.

Main screen

The patients attention have rehabilitate here



Floats: Stimulation points which are blinking. 100 times stimulation per 1 set and 1 points.



You will get 1 fish per 10 correct answer.



	ready	signal	correct answer
90%	ready	signal	push key
5%	ready	no signal	leave
5%	sleepy	signal	leave



Select 10 points and give 100 times continuous stimulation each point to the patients. When you answered correct more than 90 %, the stimulate points will be move 2mm to visual lack area.

Feedback screen 1

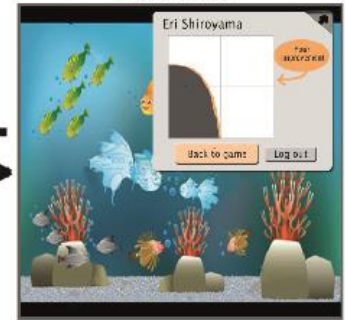
After playing 1 set of rehabilitation



Sometimes the patients can catch the new type of fish like "Catty fish". Then they can correct fishes and make own aquarium by fishing.

Feedback screen 2

After game finish



After finish the game, you will see how much the patients are improving their visual field by graphic figure.



As a interface, the patients can chose one in 4 position of interface (right-above, left-above, right-below, left below) depends on which way the patients can see very well. Also you can see the how much did you get correct answer by percentage.



When you click the right above corner, you can hide the interface and enjoy own aquarium by full screen

OO System Design Principles

- Robert C. Martin (Uncle Bob) introduced five basic principles for successful OOP and OO design
- The principles, when applied, intend to make a programmer create a system that is easy to maintain and extend over time
- Also known as the five OOD principles, the concepts are better known as the mnemonic S.O.L.I.D.

Initial	Concept
S	Single-responsibility principle
O	Open-closed principle
L	Liskov substitution principle
I	Interface segregation principle
D	Dependency Inversion Principle

Single-responsibility principle (SRP)

“A class should have only one reason to change”

- SRP states that a class should have one and only one reason to change, meaning that a class should have only one job/responsibility → in practice it is difficult to get it right
- Note that SRP has to take object context into consideration
- Example:

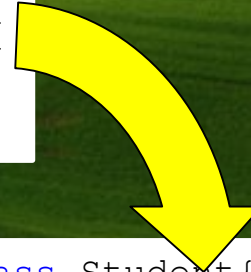
Student
- name:String - id:int
+ insertToDB(Student s):void + generateRpt(Student s):void

Two responsibilities in this class
– should be separated why?

Any changes to the methods
requires changes to Student

- To implement SRP, the Student class is split
 - Student → just to handle student info
 - StudentDB → handle insertion of student into DB
 - StudentReport → handle generation of reports from student obj.
- Any changes thus only affects the respective classes


```
public class Student{
    public string name { get; set; }
    public int id { get; set; }
    public void Insert(Student s) {
        //Database code written here }
    public void GenerateReport(Student s) {
        //Set report formatting }
}
```



```
public class Student{
    public string name { get; set; }
    public int id { get; set; }
}

public class StudentDB {
    public void Insert(Student s) {
        //Database code written here }
    public Student Select() {
        //Database code written here }
}

public class StudentReport {
    public void GenerateReport(Student s) {
        //Set report }
}
```



SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

Open-closed principle (OCP)

“Objects or entities should be open for extension, but closed from modification”

- A class should be easily extendable without modifying the class itself
- OCP can be implemented by using Abstract classes and then creating concrete classes for implementing their behavior. This will enforce having Concrete Classes extending Abstract Classes instead of changing them. e.g. Strategy Pattern.



OPEN CLOSED PRINCIPLE

Open Chest Surgery Is Not Needed When Putting On A Coat

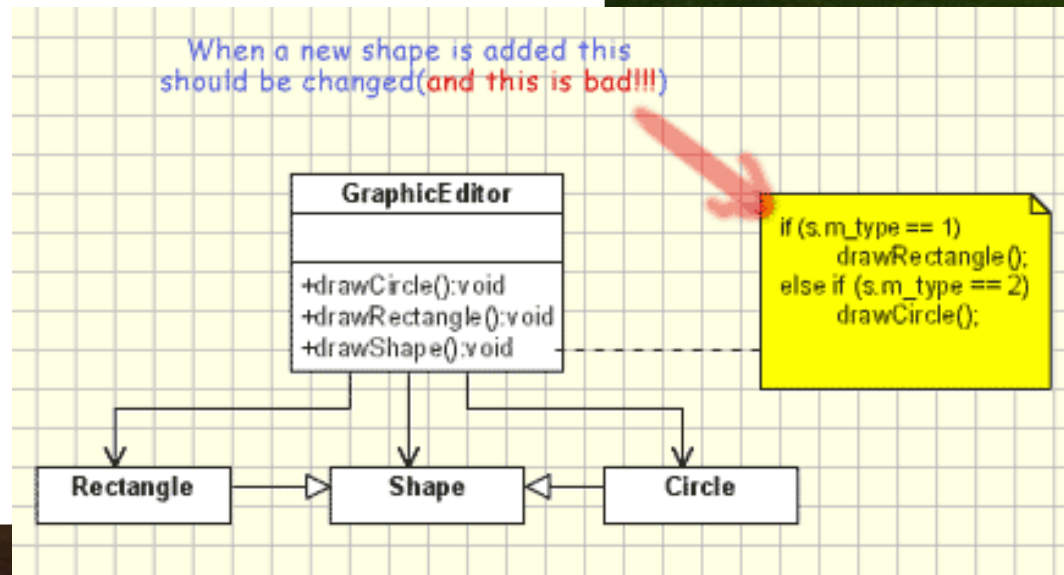

```
// Example of a class without OCP
class GraphicEditor {

    public void drawShape(Shape s) {
        if (s.m_type==1)
            drawRectangle(s);
        else if (s.m_type==2)
            drawCircle(s);
    }
    public void drawCircle(Circle r) {...}
    public void drawRectangle(Rectangle r) {...}
}

class Shape {
    int m_type;
}

class Rectangle extends Shape {
    Rectangle() {
        super.m_type=1;
    }
}

class Circle extends Shape {
    Circle() {
        super.m_type=2;
    }
}
```

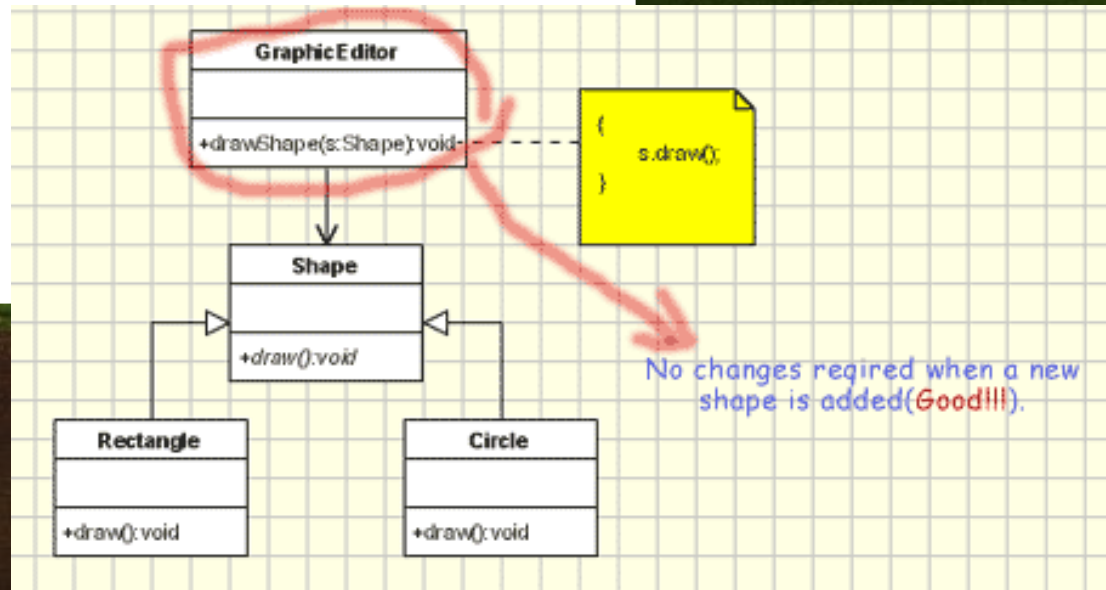


```
// Open-Close Principle - Good example
class GraphicEditor {
    public void drawShape(Shape s) {
        s.draw();
    }
}

class Shape {
    abstract void draw();
}

class Rectangle extends Shape {
    public void draw() {
        // draw the rectangle
    }
}

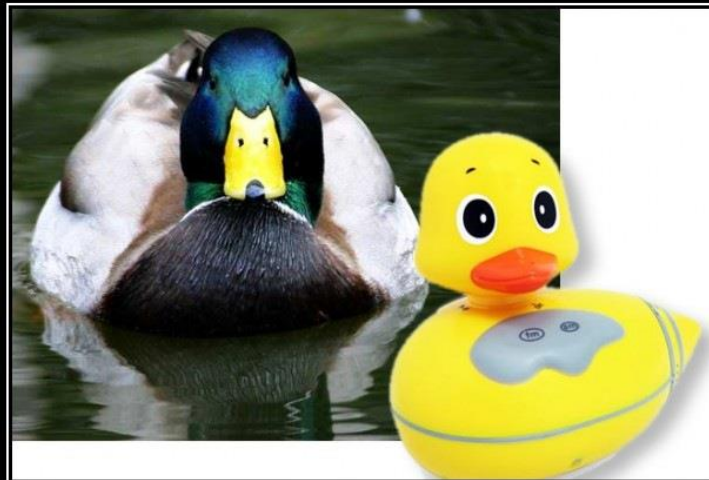
class Circle extends Shape {
    public void draw() {
        // draw circle
    }
}
```



Liskov substitution principle (LSP)

“Derived types must be completely substitutable for their base types”

- I.e. every subclass should be substitutable for their base class
- This principle is just an extension of the Open Close Principle in terms of behavior meaning that we must make sure that new derived classes are extending the base classes without changing their behavior.
- The new derived classes should be able to replace the base classes without any change in the code.



LSKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

```
class Vehicle {  
    function startEngine() {  
        // Default engine start functionality  
    }  
    function accelerate() {  
        // Default acceleration functionality  
    }  
}
```

```
class Car extends Vehicle {  
    function startEngine() {  
        $this->engageIgnition();  
        parent::startEngine();  
    }  
    private function engageIgnition() {  
        // Ignition procedure  
    }  
}
```

```
class Driver {  
    function go(Vehicle $v) {  
        $v->startEngine();  
        $v->accelerate();  
    }  
}
```


What happens if LSP is violated?

```
class Rectangle {  
    protected int m_width;  
    protected int m_height;  
    public void setWidth(int width){  
        m_width = width;  
    }  
    public void setHeight(int height){  
        m_height = height;  
    }  
    public int getArea(){  
        return m_width * m_height;  
    }  
}
```

```
class Square extends Rectangle {  
    public void setWidth(int width){  
        m_width = width;  
        m_height = width;  
    }  
    public void setHeight(int height){  
        m_width = height;  
        m_height = height;  
    }  
}
```



```
class LspTest {  
    public static void main (String args[]) {  
        Rectangle r = _____;  
  
        r.setWidth(5);  
        r.setHeight(10);  
  
        System.out.println(r.getArea());  
    }  
}
```

What happens with
the objects?

Interface Segregation Principle (ISP)

“Clients should not be forced to depend upon interfaces that they don't use”

- When we write our interfaces we should take care to add only methods that should be there.
- If we add methods that should not be there, the concrete classes implementing the interface will have to implement those methods as well even if they don't need them



Interface Segregation Principle

If IRequireFood, I want to Eat(Food food) not,
LightCandelabra() or LayoutCutlery(CutleryLayout preferredLayout)


```
interface ShapeInterface {  
    public function area();  
    public function volume();  
}
```

```
class 3DCube implements ShapeInterface {  
    public function area(){  
        // code to calculate cube surface area  
    }  
    public function volume(){  
        // code to calculate cube volume  
    }  
}
```

What about
2D
shapes?

```
class 2DSquare implements ShapeInterface {  
    public function area(){  
        // code to calculate square surface area  
    }  
    public function volume(){  
        // code to calculate square volume????  
    }  
}
```

Dependency Inversion Principle (DIP)

“High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend upon details. Details should depend upon abstractions”

- Further more it inverts the dependency: instead of writing our abstractions based on details, the we should write the details based on abstractions.

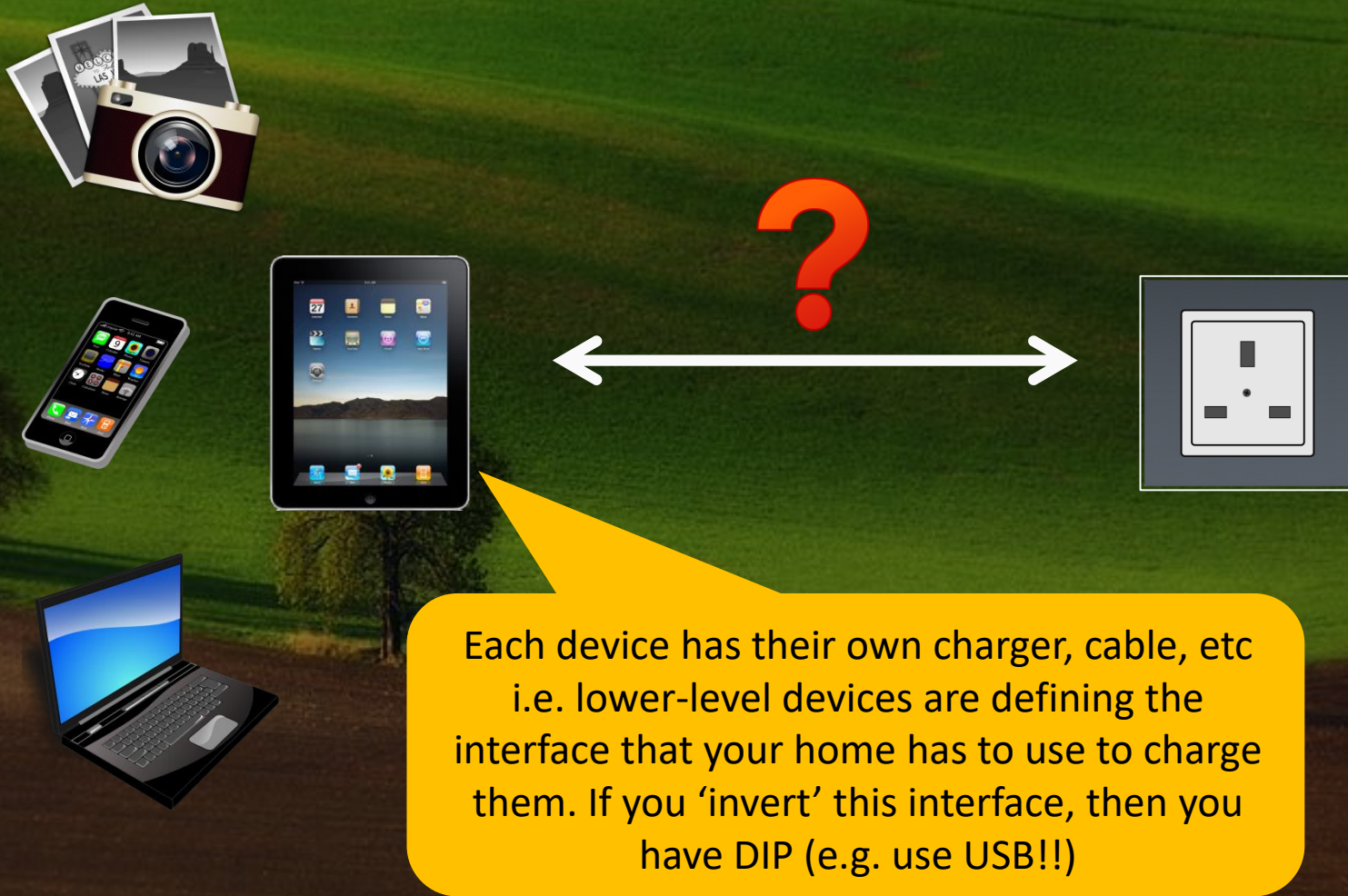


Dependency Inversion Principle

Would you solder a lamp directly to the electrical wiring in a wall?

DIP concept illustrated

- Your IT devices need charging who determines the charging mechanism (voltage, duration, plug type, connector)?



DIP concept illustrated

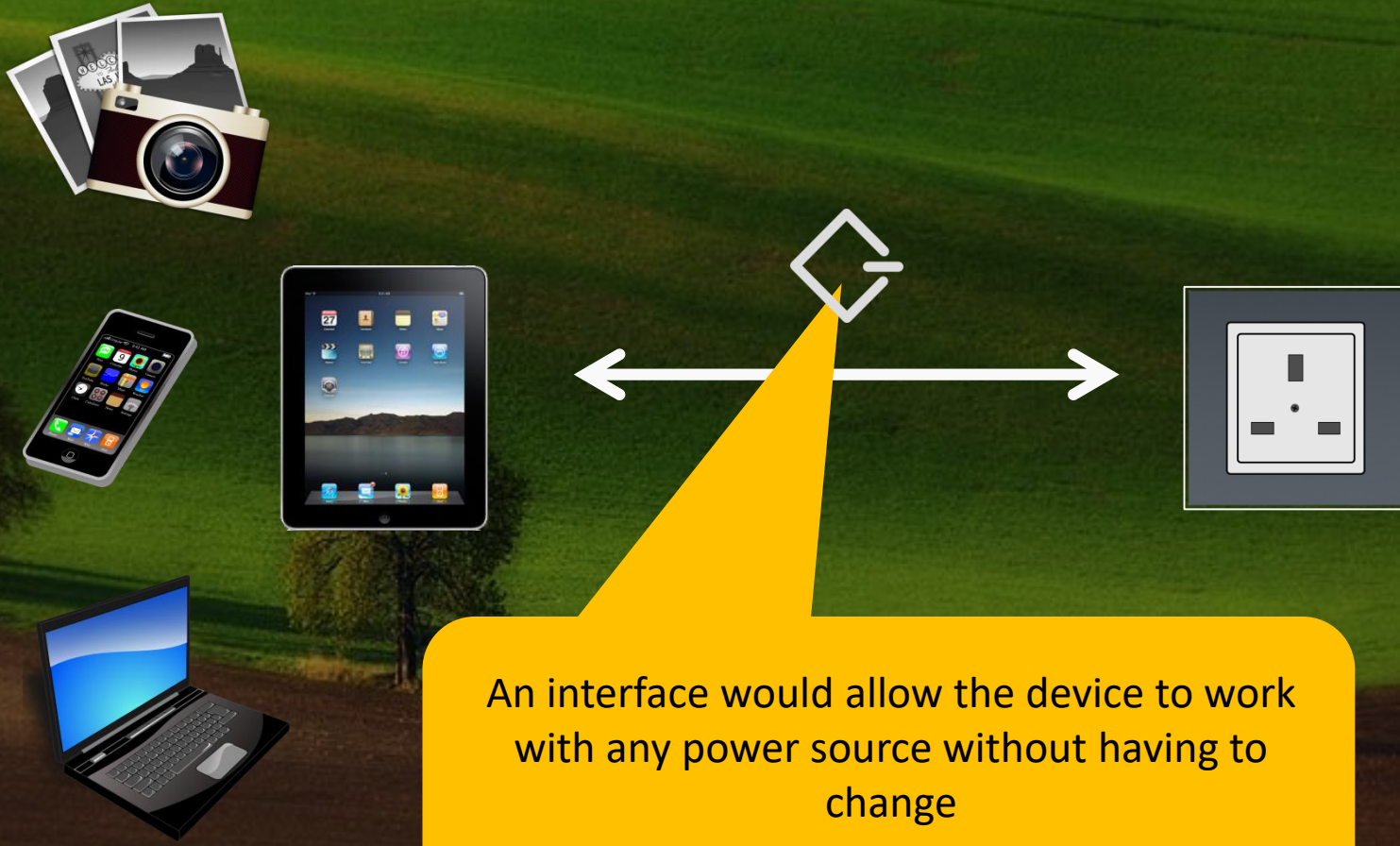
- Your IT devices need charging who determines the charging mechanism (voltage, duration, plug type, connector)?



But you can't just plug any device directly into the socket... it would blow up. So you need an interface to allow the lower-level devices to interact with higher ones

DIP concept illustrated

- Your IT devices need charging who determines the charging mechanism (voltage, duration, plug type, connector)?



DIP in code

```
class Worker {  
    public void work() {  
        // ....working  
    }  
}
```



Existing code with lower level Manager object handling the Worker

```
class Manager {  
    Worker worker;  
    public void setWorker(Worker w) {  
        worker = w;  
    }  
    public void manage() {  
        worker.work();  
    }  
}
```



```
class SuperWorker {  
    public void workhard() {  
        //.... working much more  
    }  
}
```

If a new object needs to be 'managed', the Manager has to change would this happen in real life?


```
interface IWorker {  
    public void work();  
}
```

```
class Manager {  
    IWorker worker;  
  
    public void setWorker(IWorker w) {  
        worker = w;  
    }  
  
    public void manage() {  
        worker.work();  
    }  
}
```



A better way is to implement DIP -
Here an interface to an abstract object IWorker is
created and the Manager handles all the IWorkers
regardless of type

```
class SuperWorker implements IWorker{  
    public void work() {  
        //.... working much more  
    }  
}
```



```
class LazyWorker implements IWorker{  
    public void work() {  
        //.... not working  
    }  
}
```



Difficulties and Risks in Design

- Like modelling, design is a skill that requires considerable experience
 - Individual software engineers should not attempt the design of large systems
 - Aspiring software architects should actively study designs of other systems
- Poor designs can lead to expensive maintenance - Ensure you follow as many of the principles discussed in this chapter
- It requires constant effort to ensure a software system's design remains good throughout its life
 - Make the original design as flexible as possible so as to anticipate changes and extensions.
 - Ensure that the design documentation is usable and at the correct level of detail
 - Ensure that change is carefully managed

What happens with bad design?

- For a software project
 - Creates output that
 - is difficult to verify and maintain
 - does not perform as intended in terms of functionality and performance
 - is unusable or one that is difficult to use
 - Creates irreversible bad image for the company
 - Alienates customers, both present and future
- No where can this be seen clearer than with software interfaces (which indirectly mirror the architecture of the software below)



What the customer really needed



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised



When it was delivered



What the digg effect can do to your site



The disaster recover plan

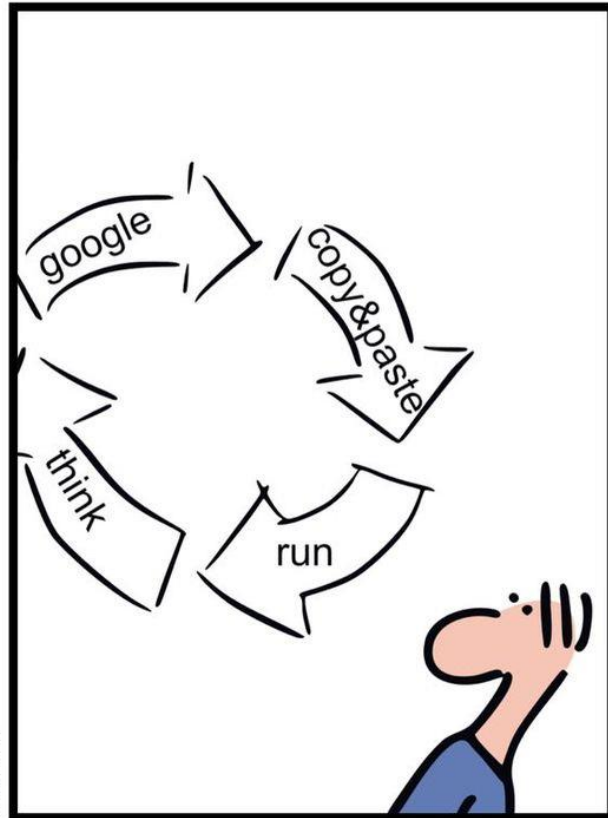
What causes bad software design?

- Social/organizational/management causes
 - Design by committee: The result of having many contributors to a design, but no unifying vision/agreement
 - Seagull management: managers only interact with employees when a problem arises, they "fly in, make a lot of noise, dump on everyone, do not solve problem, then fly out"
 - Micromanagement: Ineffectiveness from excessive observation, supervision, or other hands-on involvement from management
 - Death march: A project whose staff are compelled to continue eventhough the project is doomed for failure just because management is in denial
 - Feature creep: Uncontrolled changes in a project's scope, or adding new features to the project after the original requirements have been drafted and accepted
 - Overriding opinions: Seniority/rank rules and managers believe they are good designers because of past achievements

- Programming causes:
 - Race hazard: Failing to see the consequences of events that can sometimes interfere with each other
 - Stovepipe system: A barely maintainable assemblage of ill-related components
 - Copy and paste programming: Copying (and modifying) existing code rather than creating generic solutions
 - Reinventing the square wheel: Failing to adopt an existing solution and instead adopting a custom solution which performs much worse than the existing one
 - Interface bloat: Making an interface so powerful that it is extremely difficult to implement
 - Spaghetti code, lasagna code, boat anchoring, hard coding, coding by exception/error etc.

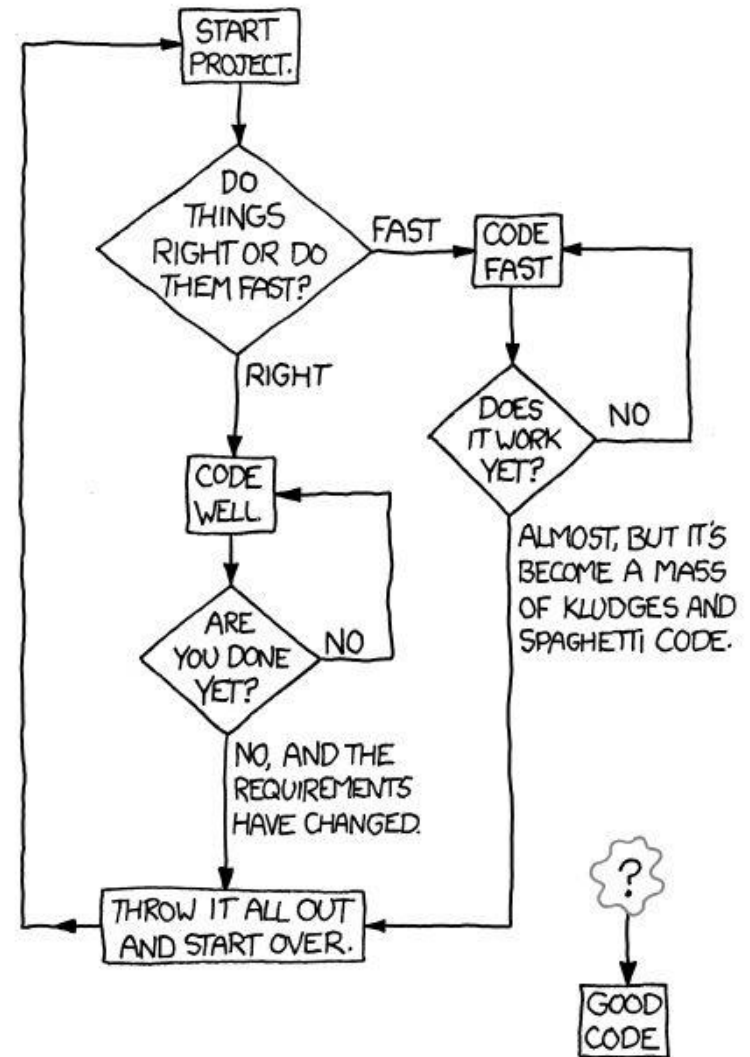
Good design $\neq$ good code

SIMPLY EXPLAINED



ACTUAL DEVELOPMENT CYCLE

HOW TO WRITE GOOD CODE:



Examples

- Microsoft's Bob lasted less than a year on the market after being ridiculed as dumbing down the Windows (then 9x) interface
- It now belongs in the bad ideas hall of fame along with Microsoft Clippy



- When first released, Windows 8's metro interface was also blasted by critics for being un-intuitive, confusing and unnecessary
- Microsoft gave in and restored Classic Desktop Mode in 8.1 and the start menu in Windows 10

AOL 1996

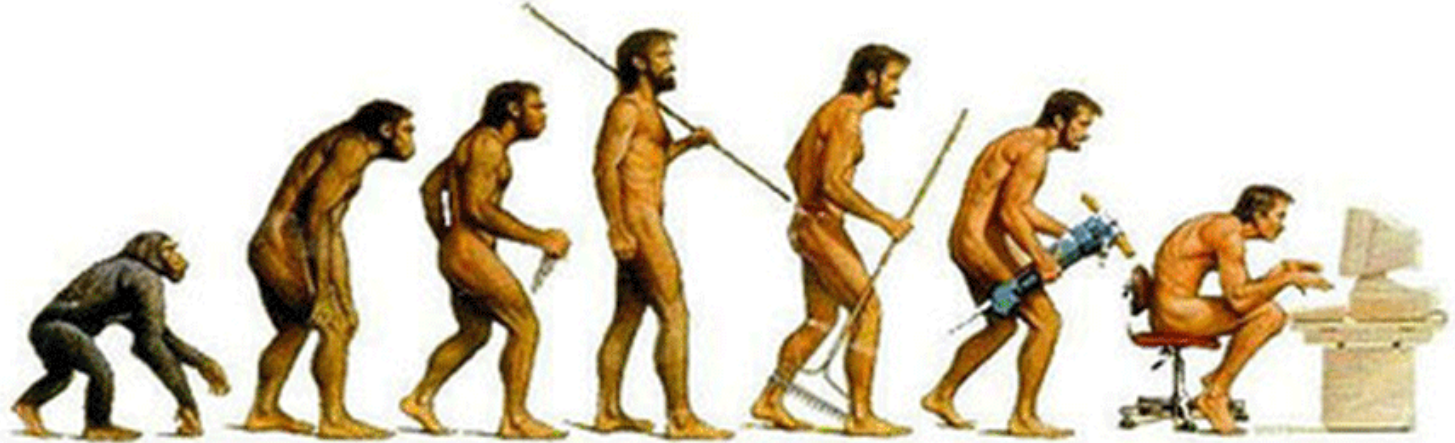


Windows 8 2012



HISTORY

Those that fail to learn from history, are doomed to repeat it.



WINDOWS 95/98

basic shapes
solid colors
no lighting



WINDOWS XP

curved shapes
simple gradients
linear lighting



WINDOWS VISTA/7

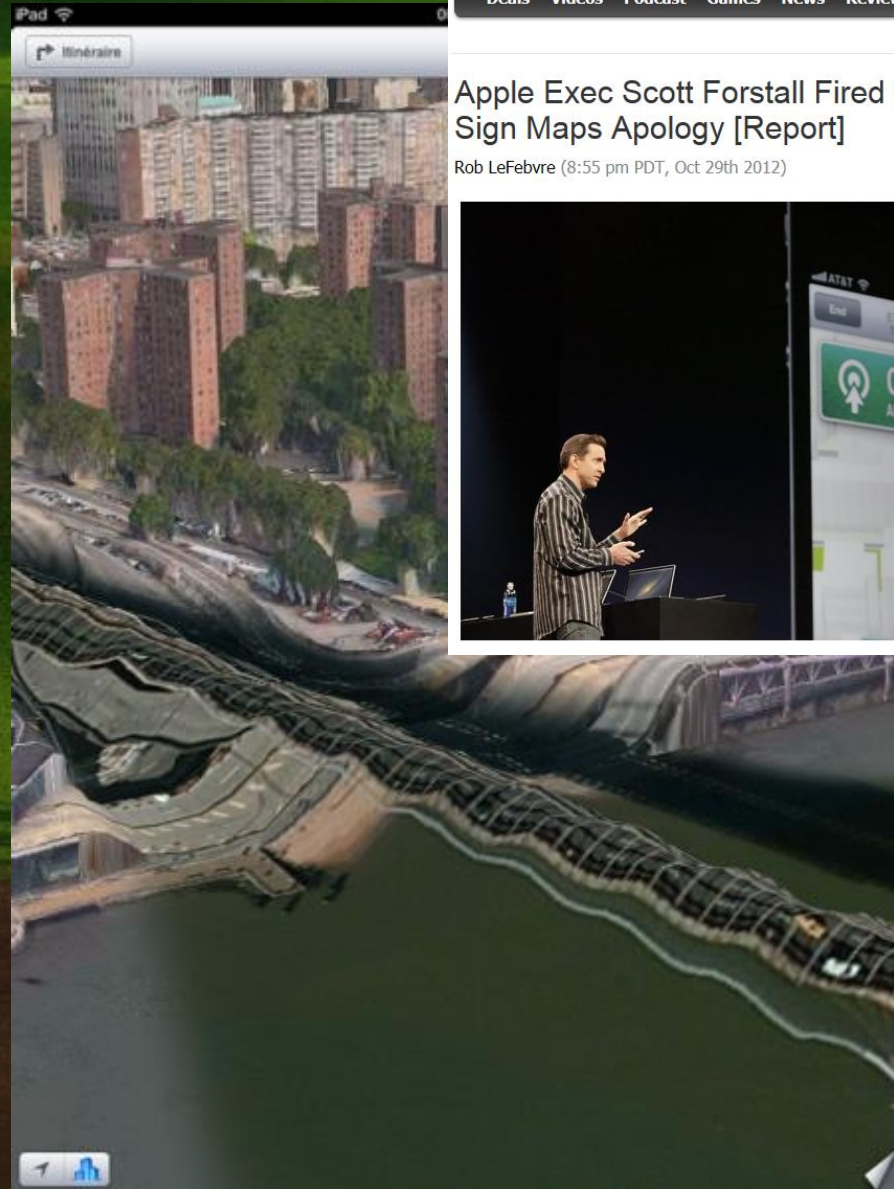
curved shapes
complex gradients
3D glow/lighting



WINDOWS 8

basic shapes
solid color
no lighting

- Apple's iOS 6 was widely praised except for its maps application which was meant to replace the default Google maps
- Widely ridiculed, it remains to this day a talking point in every iteration release of iOS (and caused the termination of its designer)



Cult of Mac



Octa-Core Smartphones
Better performance Longer lasting

- 2G RAM/16G ROM
- 3000mah Battery
- Under US \$200

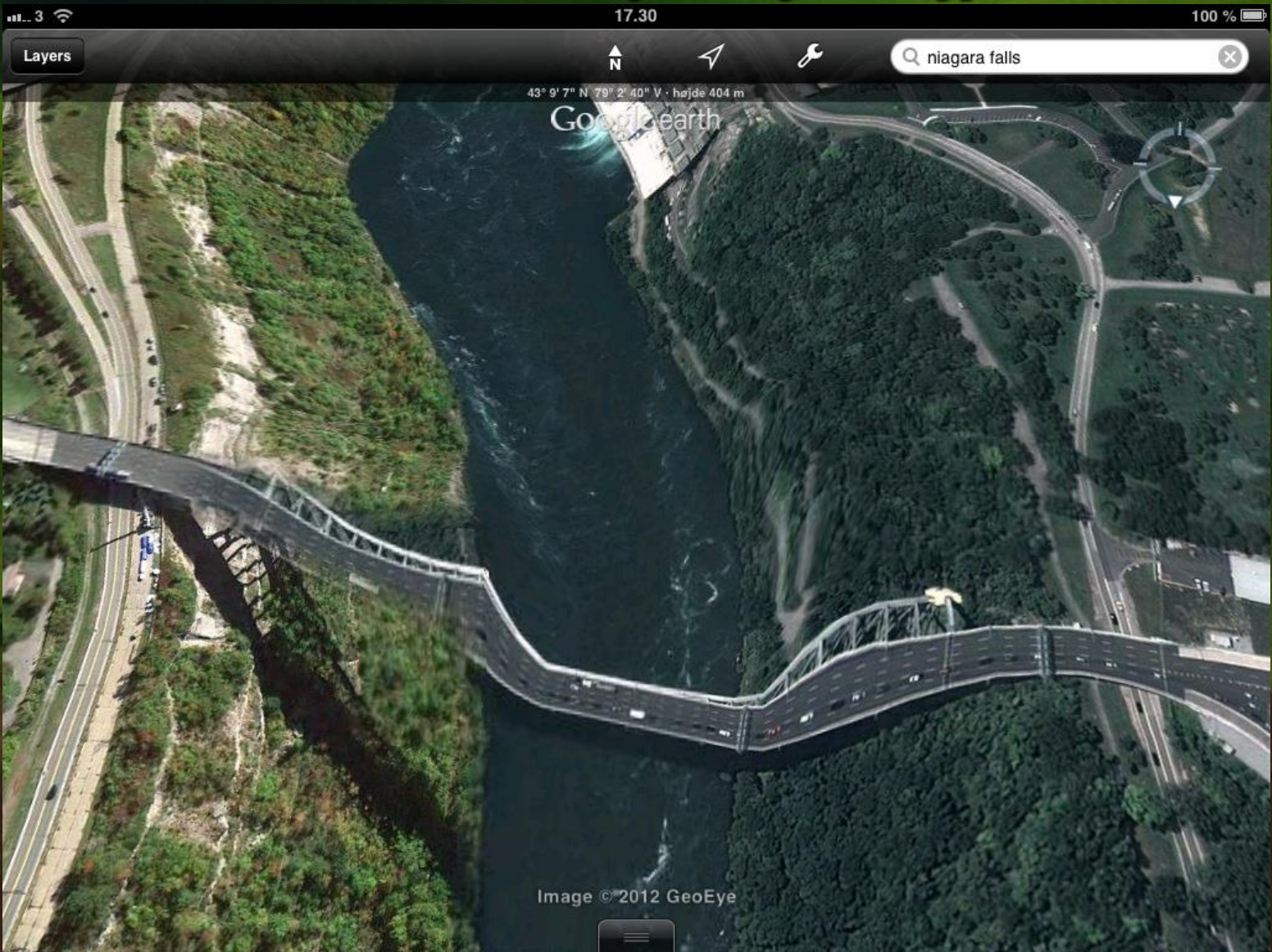
Deals Videos Podcast Games News Reviews How-tos Today's posts

Apple Exec Scott Forstall Fired Over Refusal To Sign Maps Apology [Report]

Rob LeFebvre (8:55 pm PDT, Oct 29th 2012) [Tweet](#)



- But to no surprise, Google maps had the same errors but did not suffer the same fate due to less publicity and hype



Summary

- Software architecture is a cornerstone of any software engineering project.
- The architecture describes the subsystems, the interfaces between those subsystems, as well as their dynamic behavior, interactions and shared data. It allows you to divide the system into subsystems, and to distribute the work to team members. It also allows those team members to communicate effectively, and to plan for eventual extension of the system
- All aspects of design should be communicated to readers using carefully written documentation
- Good software design will not necessarily mean good software